App Development Project Report

App Dev Project Report

1. Student Details

Name: Sravya N
Roll Number: 21F1002741
Email: 24f1002741@ds.study.iitm.ac.in
About Me: I am a dual degree student with interest in web development and data structures and algorithms.

---

2. Project Details

Project Title: placement-portal-application

Problem Statement:

To make and build a web based placement portal that helps a college handle campus placement activities between students, companies and the placement cell. The system lets companies sign up, create placement drives and hire students, while students can apply for these drives and see the status of their applications. The admin has power to control the full process by approving companies, approving placement drives and keeping track of applications

Approach:

The application is built using Flask as the backend with a simple modular setup. SQLite is used as the database and it is created directly by the application itself in the instance folder. The frontend is made using Jinja2 templates along with HTML, CSS and Bootstrap. The system has three main roles: Admin, Company and Student. Companies can sign up and create placement drives after the admin approves them. Students can register, see the approved drives and apply for them. The admin manages students, companies and placement drives, approves registrations and keeps the complete placement records of the institute.

---

3. AI/LLM Declaration

I used GitHub Copilot for assistance with minimal styling, UI enhancements, Bootstrap CSS components. AI assistance was also used occasionally while structuring some user-related logic. The overall extent of AI/LLM usage is approximately 15–20%, limited primarily to code suggestions, UI structuring, and documentation formatting. All final implementation logic, debugging, workflow integration, and system functionality were implemented and verified manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	ORM for handling database models and queries
SQLite	Lightweight local database for storing application data
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Python	Programming language used for server-side logic

---

5. Database Schema / ER Diagram

Tables:
User — stores basic account details for all users (id, username, email, password, type, blacklist_status)
Admin — stores admin-specific information linked to a user account (id, user_id)
Company — stores company profile details (id, user_id, company_name, hr_contact, website, approval_status)
Student — stores student profile information (id, user_id, first_name, last_name, phone, college, gpa, resume_path)
PlacementDrive — stores job/placement drive information created by companies (id, company_id, job_title, job_description, eligibility_criteria, application_deadline, salary, status)
Application — stores student applications for placement drives (id, student_id, drive_id, application_date, status)

Relationships:
One-to-One → User → Admin
One-to-One → User → Company
One-to-One → User → Student
One-to-Many → Company → PlacementDrive
One-to-Many → PlacementDrive → Application
One-to-Many → Student → Application



---

6. API Resource Endpoints

Endpoint	Method	Description
`/register`	GET, POST	Register new user with role selection
`/admin/dashboard`	GET	Display admin dashboard with system statistics
`/admin/companies`	GET	View company registrations and approval status
`/admin/students`	GET	View all registered students

`/admin/search`	GET	Search students or companies
`/company/create-drive`	GET, POST	Create a new placement drive
`/student/dashboard`	GET	Student dashboard with application summary
`/student/view-drives`	GET	Browse available placement drives
`/student/applications`	GET	View submitted applications
`/student/history`	GET	View application and interview history

---

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – Main Flask application entry point
- **/application/models.py** – Database models using SQLAlchemy
- **/application/controllers.py** – Flask routes for user operations and system workflows
- **/application/database.py** – SQLAlchemy database initialization and configuration
- **/application/utils.py** – Utility functions for deadline validation and status badge rendering
- **/templates** – Jinja2 HTML templates for Admin, Company, and Student interfaces
- **/static** – CSS styling and resume storage directory

Implemented Features:

- User registration and login
- Role-based access control (Admin, Company, Student)
- Placement drive creation and management
- Company approval workflow by Admin
- Student application submission and tracking
- Application status updates (Applied, Shortlisted, Selected, Rejected)
- Dashboard statistics for Admin, Company, and Student
- Resume upload and storage for student applications

- Search functionality for filtering students and companies
- Responsive UI built with Bootstrap
- Prevention of duplicate applications using database constraints
- Application deadline validation for placement drives
- Status badges for drives, companies, and applications
- Resume storage in `/static/resumes/` directory
- Role-based dashboards with separate navigation for each user type
- Error handling and success notifications for user actions
- Blacklisting functionality for students and companies

---

8. Video Presentation

Drive Link:
VID_20260311192610.mp4
